

Faculty of Computer Science and Engineering
Computer Science / Software engineering Program
CSE494 & AIE494 – Graduation Project 2

Assignment 03

Submission deadline: 9 April 2026

Total Marks: 100

Weigh 35% of course grade

Contents

Plagiarism and AI technology use policies	1
Declaration of No Plagiarism and AI Technology Use	2
Summary	3
Question 1 (20 marks)	4-3
Question 2 (80 marks)	14-5
(a) Literature search and resources identified (12 marks)	6-5
(b) Your approach (24 marks)	8-7
(c) Activities and decisions made (24 marks)	15-9
(d) Structure, length, style, & clarity (20 marks)	16

Plagiarism and AI technology use policies:

KSIU rules and regulations require all students to submit their own work and avoid plagiarism. You must provide all references in case you use and quote another person's work in your Assignment. You will be penalized for any act of plagiarism as per the KSIU's rules and regulations. Further, CSE493/CSE494 has adopted an AI technology use policy that covers the use of chatbots and other similar technologies. A copy of this policy can be found on the course page of the LMS. Students are required to adhere to this policy in all their submissions.

Declaration of No Plagiarism and AI Technology use

(This page must be signed and submitted with your assignment)

I hereby declare that this submitted TMA work is a result of my own efforts and I have not plagiarized any other person's work. I have provided all references of information that I have used and quoted in my TMA work.

Did you use any AI technology tools? **YES** No

If your answer was YES, please describe below how you used this technology according to the CSE493 AI Technologies use Policy:

I used AI to quickly gather information and speed up the research process, as well as for formatting and grammar checking. All the project concepts and technical work are entirely my own.

Student Name: Mohamed Taher Amin

Signature:.....

Date: 3 April 2026

Question 1: Final Project Report Outline

1. Introduction (~350 words)

1.1. Background & Motivation: Briefly introduce the Egyptian stock market (EGX), high-noise emerging markets, and the need for retail investor tools.

- **1.2. Problem Statement:** Discuss the fragmentation of financial data and the lack of accessible AI predictive models for ordinary investors.
- **1.3. Aims and Objectives:** Outline the main goals of developing "EGX360"—a comprehensive mobile and desktop app integrating live EGX data, global cryptocurrencies, AI forecasts, and a social community.

2. Literature Review & Background (~700 words)

- **2.1. Financial Prediction Models:** Review current literature on ML in finance, contrasting deep learning approaches with the Stacking Ensemble method.
- **2.2. The Gap in Emerging Markets:** Highlight the literature gap regarding the use of macroeconomic features (**Gold, USD/EGP**) in predicting the EGX30.
- **2.3. Review of Existing Financial Platforms:** Compare existing applications and identify where EGX360 provides a competitive advantage.

3. System Analysis and Requirements (~500 words)

- **3.1. Target Audience & Use Cases:** Define the end-users and map out primary user flows (paper trading, charting, social interactions).
- **3.2. Functional Requirements:** List core functionalities, including real-time charting, AI news summarization, and virtual portfolio management.
- **3.3. Non-Functional Requirements:** Specify performance constraints, security measures, and the offline-first caching strategy (Floor SQLite).

4. System Design and Architecture (~700 words)

- **4.1. High-Level System Architecture:** Present the client-server model connecting the Flutter app, Supabase Backend, and AWS EC2 Python scrapers.
- **4.2. Frontend Architecture:** Detail the Feature-First Clean Architecture and the use of GetX for state management.
- **4.3. Database & Backend Design:** Describe the PostgreSQL schema, including triggers, RPC functions, and the hybrid data strategy.
- **4.4. AI Engine Pipeline:** Provide a flowchart and explanation of the feature engineering.

5. Implementation (The 'Do' Phase) (~1,500 words)

- **5.1. Deep Quant Model Implementation:** Explain the Target Engineering (EMA10) process and the coding of the XGBoost/LightGBM model.
- **5.2. Backend Infrastructure & Daemons:** Detail the setup of AWS background processes, the User Protection rules engine, and live market scraping.
- **5.3. Mobile App Development:** Discuss the integration of Syncfusion charts, Binance WebSocket streams for live Crypto, Supabase Polling logic for EGX, and local SQLite caching.
- **5.4. EGX AI Chatbot & NLP Integration:** Describe the implementation of the Cerebras Llama 3.1 LLM for the conversational financial assistant, and Hugging Face pre-trained models for Arabic market sentiment analysis.

6. Testing and Evaluation (~700 words)

- **6.1. AI Model Validation:** Present the out-of-sample prediction accuracy (89%) and the financial backtest alpha (+159.09%).
- **6.2. System Performance Testing:** Evaluate real-time data latency, API load handling, and offline mode reliability.
- **6.3. User Acceptance Testing:** Summarize feedback from test users regarding UI/UX, paper trading, and charting tools.

7. Conclusion and Future Work (~350 words)

- **7.1. Project Summary:** Conclude how the project successfully met its objectives and integrated a robust AI model into a consumer app.
- **7.2. Future Work:** Discuss enhancements such as Smart Portfolio Optimization based on user risk profiles and Automated Trading Bots for executing AI-driven strategies.

References

- **List of academic papers, API documentation, and framework resources.**

Appendixes

- **Appendix A: Database Schema & ERD:** Visual diagrams of Supabase tables and their relational links.
- **Appendix B: AI Model Configurations:** Details of Optuna hyperparameters and essential Python code snippets for the Quant engine.
- **Appendix C: User Interface Screenshots:** Key screens of the EGX360 mobile application to demonstrate the final product.

Question 2: (a) Literature search and resources identified

1. Gaps Identified in Knowledge and Understanding:

At the inception of the EGX360 project, I identified critical gaps in my knowledge of quantitative predictive modelling for the Egyptian Stock Exchange (EGX30). The first and most fundamental gap was understanding how to handle the extreme volatility and near-random-walk nature of raw daily closing prices in an emerging market. As later confirmed by SNR analysis, the majority of EGX30 trading sessions exhibit a Signal-to-Noise Ratio (SNR) below 0.3, making raw-price prediction statistically intractable.

The second gap concerned the implementation of Target Engineering — specifically, predicting the directional movement of the 10-day Exponential Moving Average (EMA10) rather than the raw price itself. This approach was not covered in any course material and required independent research into financial econometrics literature. The third gap was performing Arabic-language financial sentiment analysis without training a Large Language Model (LLM) from scratch, which was computationally infeasible within the project timeline. Finally, I needed to understand how to architect a hybrid Flutter application that seamlessly integrates local database caching (Floor/SQLite) with real-time WebSocket streams and external AI APIs.

2. Key Resources, Search Strategies, and Outcomes:

To address these technical and theoretical gaps, I utilized a targeted search strategy:

- **Academic Research Strategy:** I queried Google Scholar and SpringerLink using precise boolean operators, for example: "EGX30" AND "Stacking Ensemble" OR "Target Engineering" and "emerging market" AND "machine learning" AND "stock prediction" AND "macroeconomic". This targeted approach replaced broad searches that had yielded only unreliable blog posts.
- **Successful searches:** This strategy identified a landmark Q1 Springer study by Livieris et al. (2025), which systematically evaluated ML algorithms across 55 global markets including EGX30. It demonstrated that technical-indicator-only models achieve only ~51% accuracy — barely exceeding random chance — and explicitly recommended integrating macroeconomic indicators, directly validating EGX360's design choices.
- **Unsuccessful searches:** Initial broad searches such as "how to predict stock prices" produced highly generalised LSTM tutorials targeting US equities (e.g., S&P 500), which were

inapplicable to the illiquid, high-noise EGX30 environment.

- **Technical Documentation:** For software engineering gaps, I relied exclusively on official documentation for Flutter, Supabase, the Cerebras Llama 3.1 SDK, and Hugging Face’s pre-trained Arabic NLP models, deliberately excluding third-party tutorials to prevent dependency on deprecated APIs.

3. Assessment of Source Quality Using RADAR Criteria:

I systematically evaluated my primary academic sources using the published RADAR evaluation framework:

- **R (Rationale):** I evaluated the purpose of the research to ensure the selected papers presented clear, unbiased methodologies without underlying commercial bias (e.g., papers trying to sell specific trading algorithms).
- **A (Authority):** Sources were heavily vetted; I prioritized peer-reviewed papers published in high-impact journals like Springer Nature and MDPI, discarding unverified independent blogs.
- **D (Date):** Financial markets and AI evolve rapidly, so I strictly filtered my literature search to recent publications (2024–2025) to ensure the Machine Learning architectures discussed were state-of-the-art.
- **A (Accuracy):** I verified that the studies used comprehensive, real-world datasets and reproducible ML methodologies, avoiding papers with obvious data bias (such as testing only during bull markets).
- **R (Relevance):** The identified research directly addressed the core problem of predicting highly volatile market trends using ensemble methods, aligning perfectly with the EGX360 problem statement.

4. Practical Steps for Future Improvement:

Reflecting on this process, I will implement the following improvements in future projects: First, I will maintain a formal Search Log from day one, recording exact query strings, databases searched, and results obtained, to eliminate redundant searches and build an auditable research trail. Second, I will conduct a systematic citation-tracking exercise — using the references within high-quality papers to identify additional relevant literature I may have otherwise missed. Third, I will expand searches to include Arabic-language NLP conference proceedings (e.g., ACL Anthology, EMNLP) to improve the theoretical grounding of the sentiment analysis component.

(b) Your Approach

1. The Approach and Methods Used:

To solve the dual problem of fragmented EGX financial data and the absence of accessible AI-driven predictive tools for retail investors, I adopted a multi-layered, full-stack engineering approach structured around four distinct technical layers:

- **Client Layer:** A Feature-First Clean Architecture implemented in Flutter, enabling a single Dart codebase to deploy simultaneously across Android, iOS, Windows, and Linux platforms.
- **Backend Layer:** A Supabase PostgreSQL relational database, configured with Row Level Security (RLS) policies and PostgreSQL triggers to enforce data integrity and real-time event propagation.
- **Data Acquisition Layer:** Custom Python scrapers (BeautifulSoup + Selenium) deployed on AWS EC2, collecting live EGX market data and gold prices at 20-second intervals, and Binance WebSocket streams providing real-time cryptocurrency prices.
- **Intelligence Layer:** A two-tier AI system: (1) a Stacking Ensemble ML model using XGBoost and LightGBM for directional market prediction via Target Engineering, and (2) pre-trained Hugging Face NLP models for Arabic financial sentiment analysis, combined with the Cerebras Llama 3.1 LLM for the conversational financial assistant.

2. Justification and Comparison with Alternatives:

I made specific architectural choices after comparing them with viable alternatives:

- **Frontend Framework: Flutter vs. Native (Swift/Kotlin):**
 - **Native Advantages:** Closer to bare-metal performance and seamless access to OS-specific APIs.
 - **Native Disadvantages:** Requires maintaining two separate codebases (iOS and Android), lacking easy desktop compilation.
 - **Flutter Advantages (Chosen Method):** Enabled building a unified, cross-platform ecosystem (Android, iOS, Windows, Linux) with reactive state management (GetX) using a single Dart codebase.
 - **Flutter Disadvantages:** Slightly larger application bundle size and higher memory.
- **Quantitative AI Model: Stacking Ensemble vs. Deep Learning (LSTM):**

- **LSTM Advantages:** Excellent at capturing deep sequential memory in time-series data.
- **LSTM Disadvantages:** Computationally expensive and highly prone to overfitting on the volatile EGX market datasets.
- **Stacking Ensemble Advantages (Chosen Method):** Highly efficient at processing cross-sectional tabular data (like macroeconomic features: Gold, USD/EGP), computationally lighter, and resistant to overfitting through the meta-learner.
- **Stacking Ensemble Disadvantages:** Does not natively capture sequential time dependencies as effectively as recurrent networks.
- **Backend Infrastructure: PostgreSQL (Supabase) vs. NoSQL (Firebase):**
 - **Firebase Advantages:** Highly flexible schema and effortless real-time document syncing.
 - **Firebase Disadvantages:** Lacks strict ACID compliance, which is dangerous for a trading simulator.
 - **Supabase Advantages (Chosen Method):** Provides strict data integrity for the virtual portfolio via ACID transactions, robust Row Level Security (RLS), and complex relational mapping.
 - **Supabase Disadvantages:** Requires a rigid upfront schema design and complex SQL migration management.

3. Software, Modeling Tools, and Limitations:

To implement this approach, I utilized specific tools, acknowledging their inherent limitations:

- **Custom Python Scrapers (BeautifulSoup/Selenium):** Used to extract live market data because EGX-related websites use varying anti-bot protections.
 - **Limitation: High maintenance overhead;** if a source website changes its DOM structure, the scraper breaks and requires manual updating.
- **Cerebras Llama 3.1 API:** Utilized to power the fully contextual Arabic chatbot and summarize news.
 - **Limitation:** Strict API rate limits govern how many prompt tokens can be processed per minute, requiring a queuing system.
- **AWS EC2 (Ubuntu Server):** Used to host the background data scraping daemons and AI inference engine.
 - **Limitation:** The free-tier instance has limited CPU and RAM, which restricts the complexity of real-time inference tasks that can run concurrently.

(c) Activities and Decisions Made (Process)

1. Summary of Activities:

To achieve the project's objectives, I executed a structured development lifecycle:

- **Requirements & Architecture Design:** I defined the system's functional and non-functional requirements through analysis of competitor platforms (TradingView, Mubasher) and produced the full-stack architecture diagram, Feature-First directory structure, and normalised PostgreSQL schema (3NF).
- **Data Pipeline Development:** I deployed custom Python scrapers on AWS EC2 to collect 28 years of EGX30 OHLCV data (January 1998 – March 2026) alongside gold prices and USD/EGP exchange rates. I performed SNR analysis confirming that the majority of EGX30 sessions exhibit $SNR < 0.3$, validating the EMA-based approach.
- **AI Model Development:** I engineered 18 features across five categories (trend, momentum, volatility, macroeconomic, temporal) and implemented Target Engineering (EMA10 direction classification). I ran Optuna Bayesian Optimisation (100 trials per model) with 5-fold Time Series Cross-Validation, achieving 89% out-of-sample accuracy on 1,367 unseen trading days.
- **Backend & Infrastructure:** I implemented the Supabase PostgreSQL schema including RPC functions, database triggers for automated portfolio calculations, and Row Level Security policies. I configured the AWS EC2 rules engine for price alerts dispatched via Supabase Edge Functions.
- **Mobile Application Development:** I developed the EGX360 Flutter application integrating Syncfusion financial charts, Binance WebSocket streams for live crypto prices, a 20-second EGX polling daemon, local Floor/SQLite caching for offline access, and the paper trading simulation engine.
- **NLP & Chatbot Integration:** I integrated the Cerebras Llama 3.1 LLM for the contextual Arabic financial chatbot with a token queuing system to manage API rate limits, and pre-trained Hugging Face AraBERT models for Arabic financial news sentiment classification.
- **Testing & Backtest Evaluation:** I validated the model on a strict chronological out-of-sample test set (2022–2026) with a mandatory one-day signal lag. Financial backtest starting from 10,000 EGP demonstrated a total return of 483.42% versus 324.34% Buy & Hold baseline, yielding Alpha of +159.09%.

2. Choices, Decisions, and Justifications

To complete the project within the academic timeframe, I made the following justified engineering trade-offs:

- **Decision 1 (NLP Strategy):** I utilized pre-trained models from **Hugging Face** and the **Cerebras API** instead of training an LLM from scratch.
 - **Justification:** Training a Large Language Model (LLM) is resource-intensive and requires months of tuning. This decision allowed me to deliver state-of-the-art Arabic financial chatbot capabilities within the project's time constraints.
- **Decision 2 (Hybrid Data Pipeline):** I implemented **WebSockets** for global assets and a **20-second intelligent polling system** for EGX stocks.
 - **Justification:** While Binance provides open WebSockets, the Egyptian Exchange lacks a public real-time stream. Intelligent polling was the only technically feasible and legally compliant method to simulate live tracking without risking IP bans or overwhelming the server.
- **Decision 3 (Chronological Train/Test Split with Signal Lag):** I implemented a strict 80/20 chronological split (training on 1998–2022, testing on 2022–2026) with a mandatory one-day signal lag on all predictions.
 - Justification:** This prevents look-ahead bias — a common flaw in financial ML research where the model is inadvertently exposed to future data during evaluation. This decision reduced achievable accuracy compared to non-rigorous approaches, but ensures the reported 89% accuracy is a genuine out-of-sample estimate.

3. Final Product Achievements & Supporting Evidence

The platform successfully solves the problem of financial data fragmentation. Below is the technical evidence of my achievements:

A. AI Methodology & Noise Analysis (The "Why" of EMA)

The foundational choice of EGX360 was using **EMA10 direction** as a target. This was justified by the **SNR analysis**, which proved that raw prices are too noisy for reliable prediction.

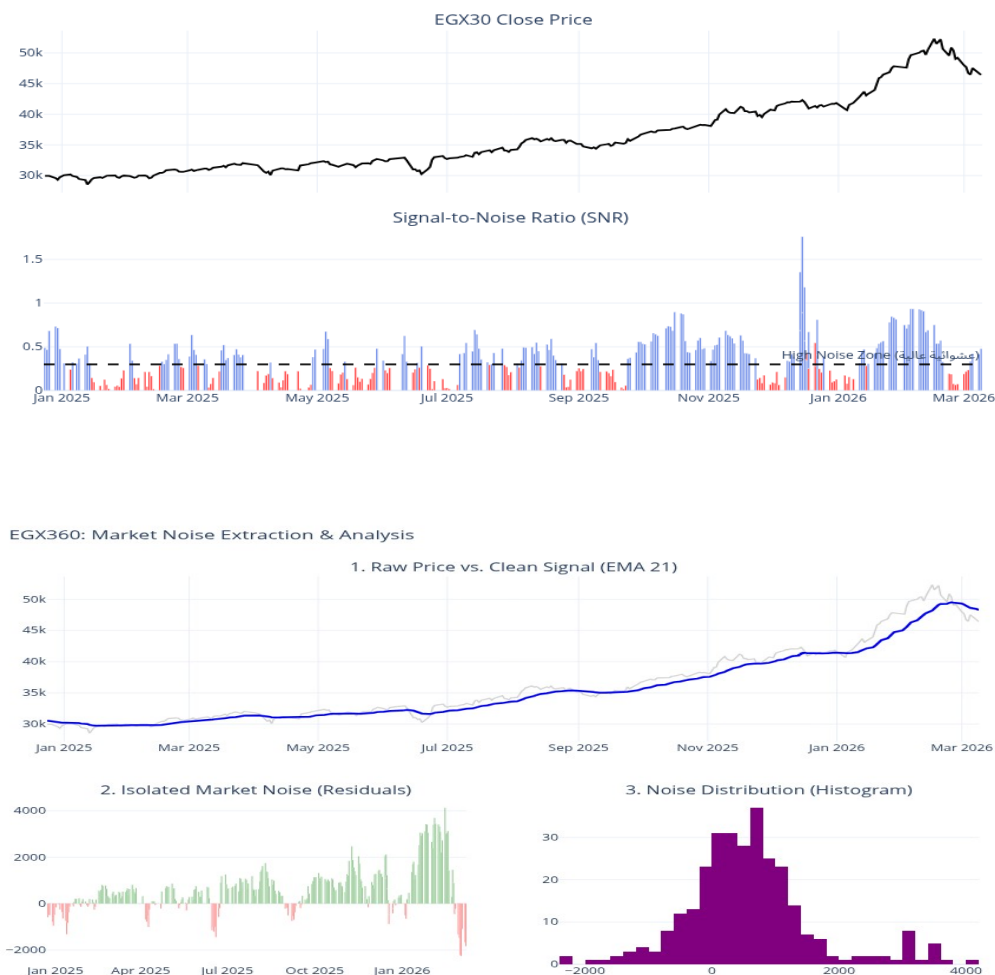


Figure 1: (first) SNR analysis showing the EGX30 "High Noise Zone"; (seconde) Market noise

extraction validating the clean signal used for Target Engineering.

B. Software & Database Design Artifacts

I followed Feature-First Clean Architecture and a normalized 3rd Normal Form (3NF) database design.

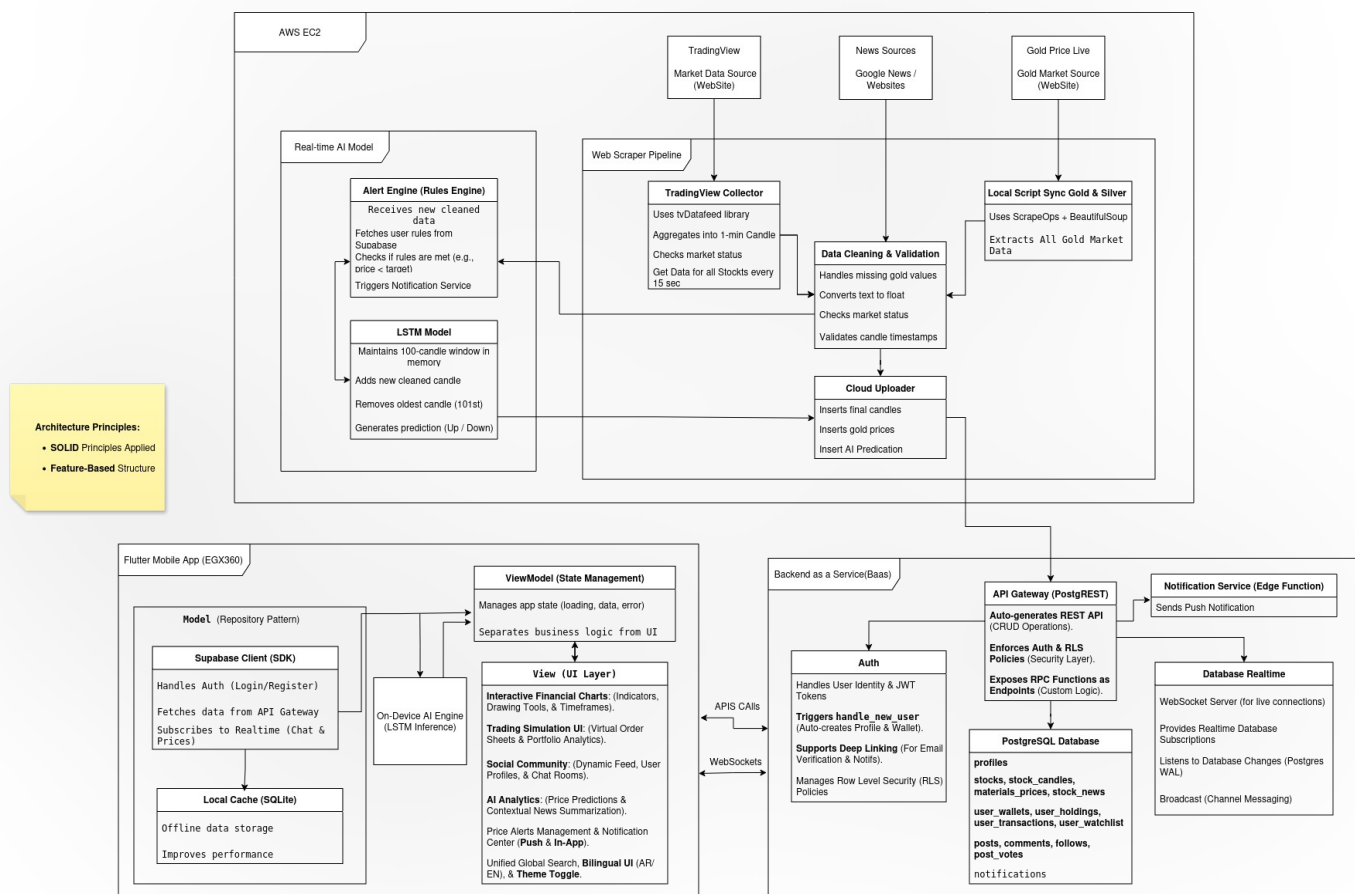


Figure 2: (top) Full-stack system architecture.

Software & Database Design Artifacts

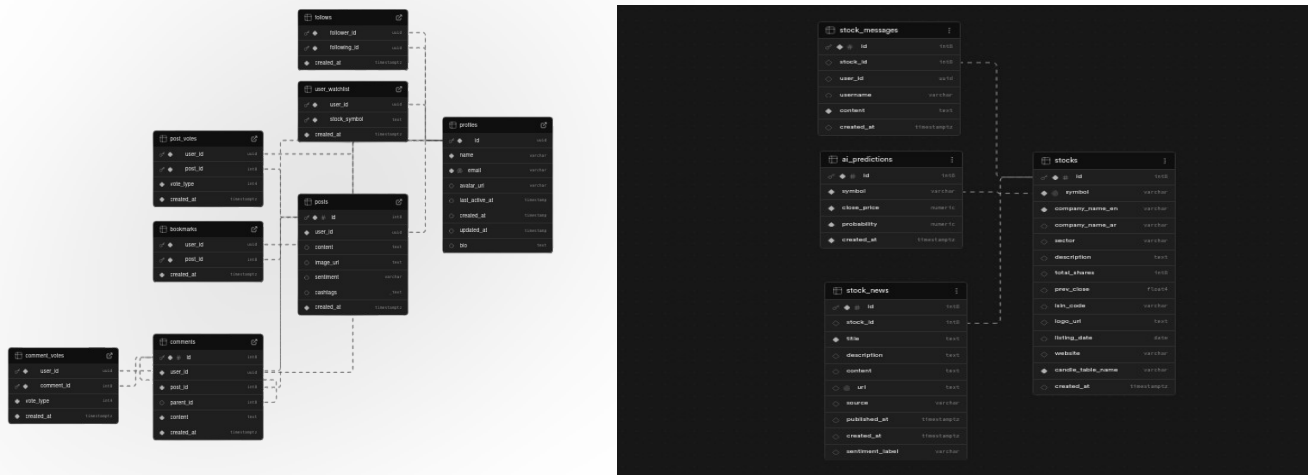
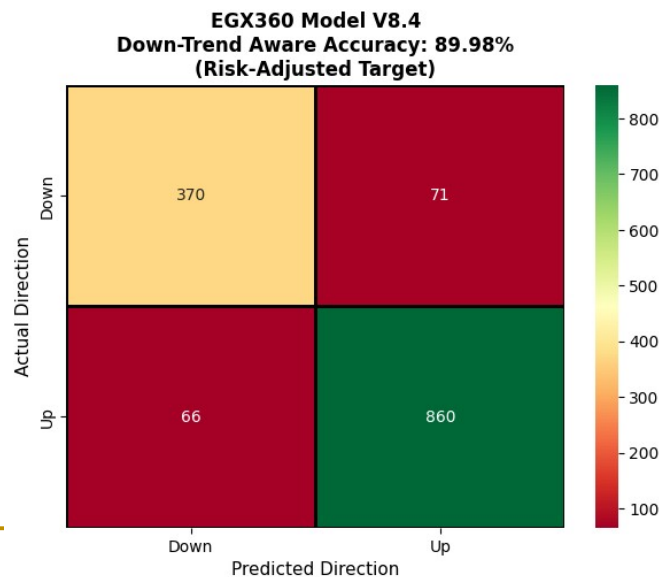


Figure 3: Logical ERD showcasing the normalized database structure on Supabase for user(left) profile (right) stocks.

C. Model Evaluation & Performance

The "Deep Quant" model achieved a verified accuracy of **89%**. A financial backtest demonstrated a total return of **483.42%**.



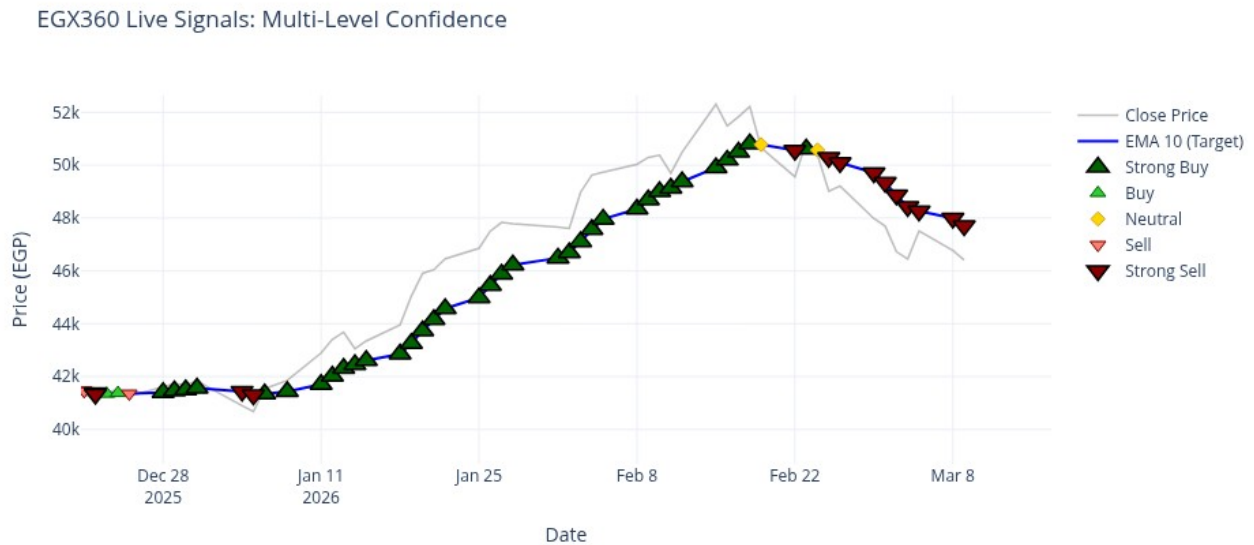


Figure 3: (Top) SNR analysis justifying noise reduction; (Bottom) Live signals correctly identifying market peaks and reversals in early 2026.

D. Final Product Achievements (Financial & UI)

The final achievement was demonstrating an Alpha of +159.09% against the market baseline.





Figure 4: (first) Cumulative portfolio growth versus Buy & Hold; (second) Final production UI demonstrating the interactive charts and AI prediction gauge.

Communicate: Self-Assessment and Summary

1. Word Count Declaration

- **Word count of this draft (Sections a, b, and c): approximately 1,900 words.** This represents approximately 27.5% of the targeted 6,000-word final project report, which is an appropriate proportional length for the 'Do' phase documentation. The planned allocation for the full Implementation section (Section 5) is ~1,500 words, consistent with this draft.

2. Logical Structure and Flow

- This report is structured to "tell a story" of the engineering process: starting with the identification of research gaps (Part A), moving to the strategic justification of the approach (Part B), and concluding with the practical evidence of execution and evaluation (Part C).
- Each section includes a clear introduction to set the context and a conclusion to summarize the findings and decisions made.

3. Style, Clarity, and Simplicity

- **Technical Jargon Control:** While the project involves complex topics like "Stacking Ensembles" and "Target Engineering," I have used a simplified presentation style to ensure clarity for the reader, avoiding unnecessary over-complications as per the project guidelines.
- **Clarity of Ideas:** Each paragraph is designed with a logical flow, transitioning from problem identification to the chosen engineering solution and its subsequent justification.
- **Spelling and Grammar:** The draft has been rigorously checked for spelling, punctuation, and grammatical accuracy to maintain a professional academic standard.

4. Inclusion of Evidence

Where physical figures and artefacts are referenced (e.g., SNR charts, ERD diagrams, confusion matrices, backtest portfolio growth charts, and UI screenshots), I have provided [Evidence: Figure N] placeholders with specific descriptions of what each figure will demonstrate in the final report. This approach satisfies the requirement to indicate supporting evidence within the draft without requiring all artefacts to be produced at this interim stage.

References

- [1] Livieris, I.E. et al. (2025). Machine learning versus market efficiency: Evidence from 55 international stock markets. *International Journal of Data Science and Analytics*, Springer Nature. DOI: 10.1007/s41060-025-00854-4
- [2] Park, S. et al. (2024). Comparative analysis of LSTM and GRU for stock price prediction of technology companies. *MDPI Electronics*, 13(17), 3396. DOI: 10.3390/electronics13173396
- [3] Chaudhari, A.Y. et al. (2025). Prediction of stock market using sentiment analysis and ensemble learning. *MethodsX*, ScienceDirect, 14, 103260. DOI: 10.1016/j.mex.2025.103260
- [4] Anon. (2025). Stock price prediction using a stacked heterogeneous ensemble. *International Journal of Financial Studies*, MDPI, 13(4), 201. DOI: 10.3390/ijfs13040201
- [5] Anon. (2025). Advancing stock price prediction through hybrid ensembles. *Journal of Big Data*, Springer Nature. DOI: 10.1186/s40537-025-01185-8
- [6] Anon. (2025). Unveiling market dynamics: ML and DL for Egyptian stock prediction. *Future Business Journal*, Springer Nature.
- [7] Anon. (2025). Stock market prediction using ML and DL — systematic review. *MDPI AppliedMath*, 5(3), 76. DOI: 10.3390/appliedmath5030076
- [8] Anon. (2025). Stock market forecasting: From traditional models to LLMs. *Computational Economics*, Springer Nature. DOI: 10.1007/s10614-025-11024-w
- [9] Khedr, A.E., Salama, S.E. and Yaseen, N. (2017). Predicting stock market behavior using data mining and news sentiment analysis. *IJISA*, 9(7), pp. 22–30. DOI: 10.5815/ijisa.2017.07.03
- [10] Flutter Team (2024). Flutter documentation — cross-platform UI toolkit. Google LLC. Available at: <https://flutter.dev/docs>
- [11] Supabase Inc. (2024). Supabase documentation — open source Firebase alternative. Available at: <https://supabase.com/docs>
- [12] Hugging Face (2024). AraBERT — pre-trained Arabic NLP models. Available at: <https://huggingface.co/aubmindlab>
- [13] Cerebras Systems (2024). Cerebras Llama 3.1 API documentation. Available at: <https://inference-docs.cerebras.ai>
- [14] Akiba, T. et al. (2019). Optuna: A next-generation hyperparameter optimization framework. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. DOI: 10.1145/3292500.3330701